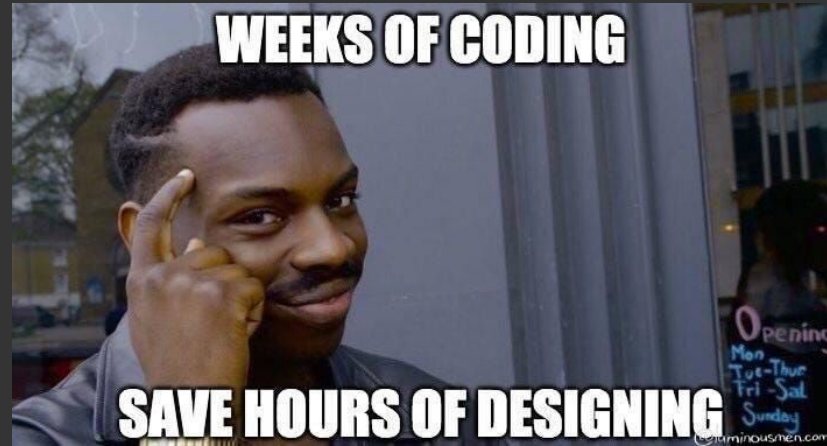


Object Oriented Analysis and Design

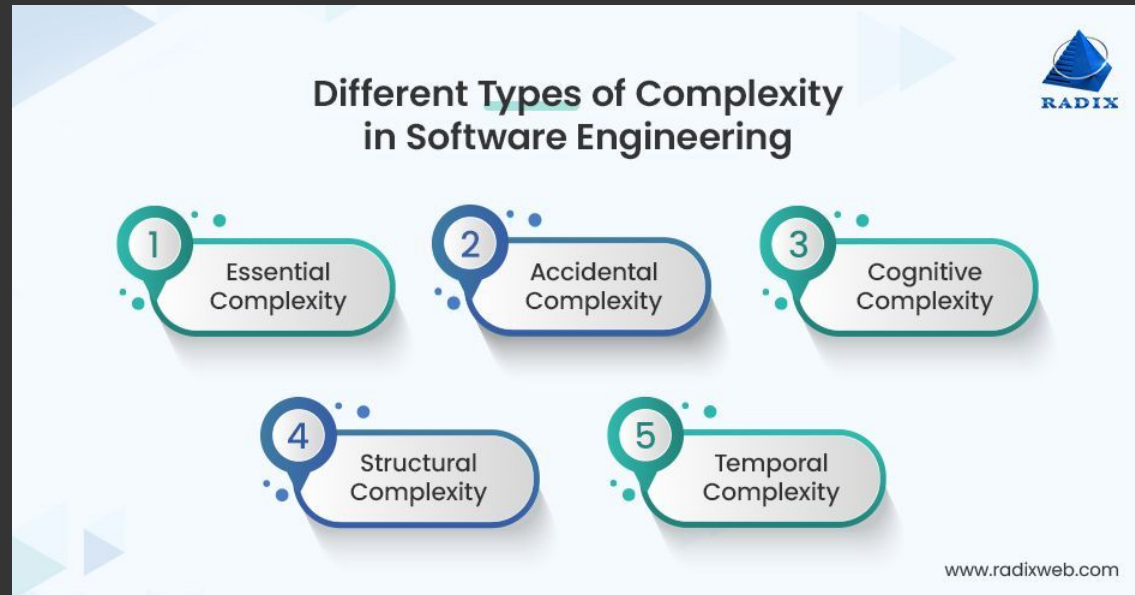
Software Design and OOP Principles

Vladimir Soroka



Analysis & Design

- Software development is a complex task



Essential Complexity

- **Definition:** This refers to the inherent difficulty of the problem being solved, regardless of how it's implemented.
- **Example:** If you're writing a program to simulate a nuclear reactor, the underlying physics and safety rules make the task complex by nature.
- **Key Point:** It cannot be removed—only managed or simplified in how it's presented.



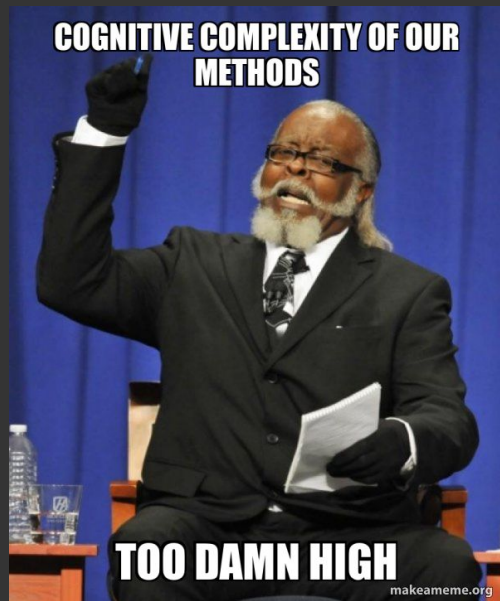
Accidental Complexity

- **Definition:** This is the complexity introduced by tools, languages, frameworks, or poor design, not by the problem itself.
- **Example:** A simple feature takes 100 lines of code because of verbose syntax or outdated frameworks.
- **Key Point:** It can be reduced through better tooling, abstraction, or modern practices.



Cognitive Complexity

- **Definition:** This relates to how hard the code is to understand for humans - developers, reviewers, or maintainers.
- **Example:** Nested loops, non-intuitive variable names, or overly clever logic.
- **Key Point:** High cognitive complexity makes systems harder to maintain and more error-prone.



Structural Complexity

- **Definition:** This refers to how complex the architecture or relationships in the system are.
- **Example:** Circular dependencies between modules, tight coupling, or overly deep inheritance trees.
- **Key Point:** Good architecture (e.g., using design patterns, modularization) can minimize this.



Temporal Complexity

- **Definition:** This deals with the timing and sequencing of operations, especially in concurrent or asynchronous systems.
- **Example:** Race conditions, deadlocks, or needing to understand event ordering.
- **Key Point:** Temporal complexity is difficult to test and often leads to subtle bugs.



Software development

development

- Development Complexity:
 - Functional requirements
 - Non functional requirements
 - Gaps between the users and the developers
 - Different perspectives and assumptions

Software development

development

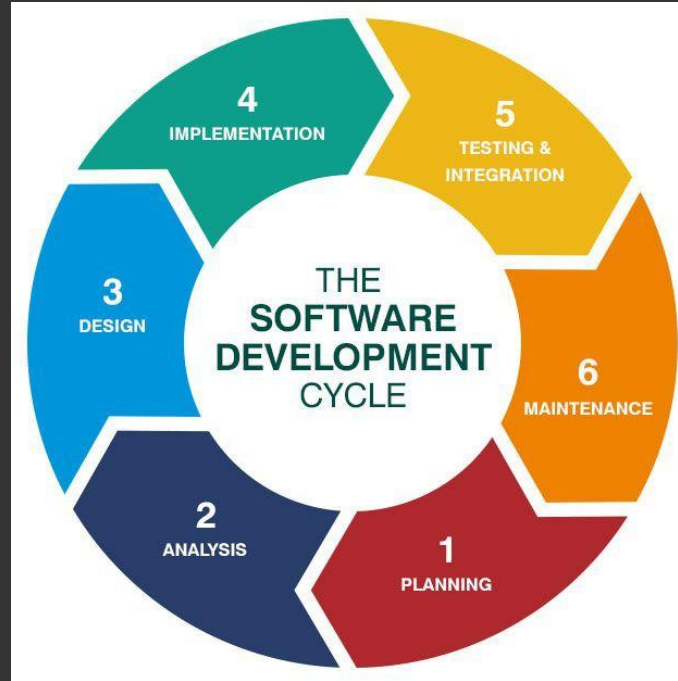
- Management complexity:
 - Enormous amount of code
 - Multiple dependencies and modules
 - Development teams coordination
 - Guiding the new developers
 - Flexibility and wide range of opportunities

Software development

development

- Software characteristics:
 - No assumable connection between operations
 - The problem field is very wide and hard to expect every option
 - A lot of edge cases that we have to check
 - Define acceptable base-line of confidence

Development life cycle



Why Do we Need Good Design?



There are two ways of constructing a software design: One way is to make it so simple that there are obviously no deficiencies, and the other way is to make it so complicated that there are no obvious deficiencies. The first method is far more difficult.

— Tony Hoare —



Why Do we Need Good Design?



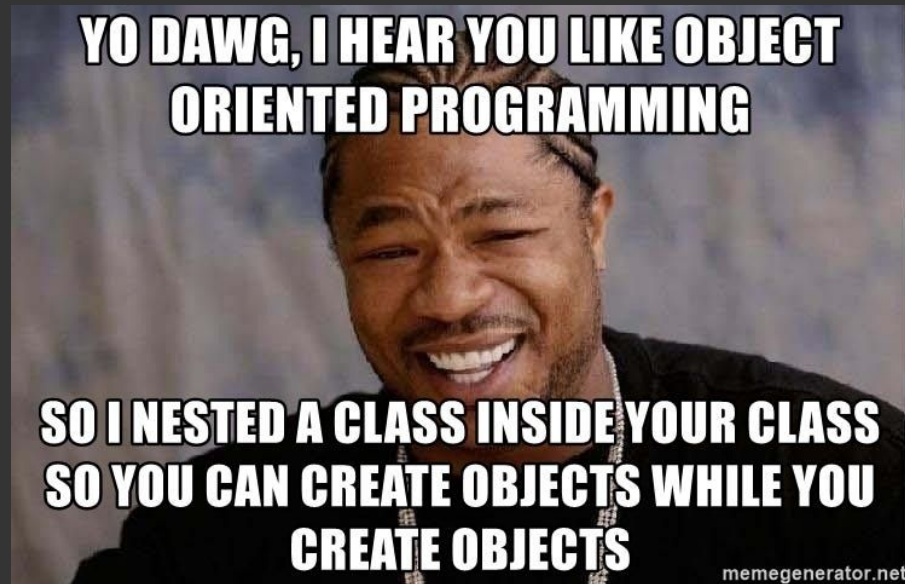
Designing software is hard, so it's unlikely that your first thoughts about how to structure a module or system will produce the best design. You'll end up with a much better result if you consider multiple options for each major design decision: **design it twice.**

John Ousterhout

 quote fancy

Object Model

- Abstraction
- Encapsulation
- Modularity
- Hierarchy



Abstraction

- **What it is:** Hiding internal details and showing only the relevant features of an object.
- **Purpose:** Simplifies complex systems by focusing only on what an object does, not how it does it.
- **Example:** A Car class exposes start() and drive() but hides how the engine works.



Abstraction

```
// Abstract base class
class Car {
public:
    virtual void start() = 0; // pure virtual function
    virtual void drive() = 0; // pure virtual function
    virtual ~Car() {}
};

// Concrete class
class Tesla : public Car {
public:
    void start() override {
        cout << "Tesla is starting silently..." << endl;
    }

    void drive() override {
        cout << "Tesla is driving in autopilot mode!" << endl;
    }
};

int main() {
    Car* myCar = new Tesla();
    myCar->start(); // Abstracted usage
    myCar->drive(); // No knowledge of internals required

    delete myCar;
    return 0;
}
```


Encapsulation

- **What it is:** Bundling data (attributes) and behavior (methods) into a single unit (class) and restricting access to internal states.
- **Purpose:** Protects the object's integrity by preventing unauthorized access or modification.
- **Example:** Private fields in a class that are accessed or modified only via public methods.



Encapsulation

```
class BankAccount {
private:
    double balance; // Private data: cannot be accessed directly from outside

public:
    BankAccount(double initialBalance) {
        if (initialBalance >= 0)
            balance = initialBalance;
        else
            balance = 0;
    }

    void deposit(double amount) {
        if (amount > 0)
            balance += amount;
    }

    void withdraw(double amount) {
        if (amount > 0 && amount <= balance)
            balance -= amount;
        else
            cout << "Insufficient funds or invalid amount." << endl;
    }
}
```

```
double getBalance() const {
    return balance;
}

};

int main() {
    BankAccount account(1000);

    account.deposit(500);
    account.withdraw(200);

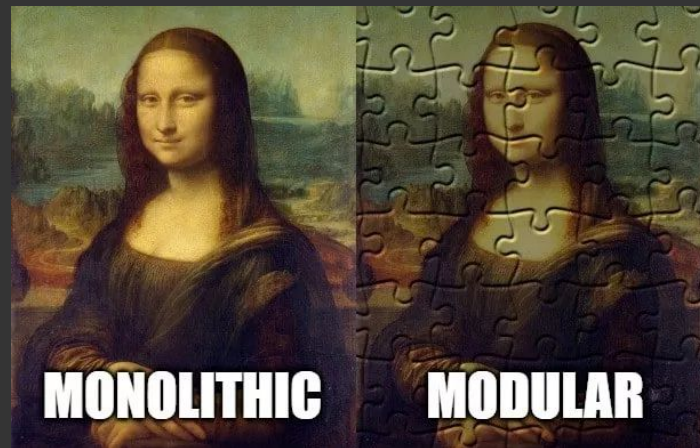
    cout << "Current balance: $" << account.getBalance() << endl;

    // account.balance = 100000; // ❌ Not allowed: balance is private

    return 0;
}
```

Modularity

- **What it is:** Dividing the software into independent, interchangeable modules (classes or components).
- **Purpose:** Makes systems easier to develop, test, maintain, and reuse.
- **Example:** A system has separate modules like User, Order, and Payment, each handling specific responsibilities.



Modularity

```
#ifndef USER_H
#define USER_H

#include <string>
using namespace std;

class User {
private:
    string username;
    string password;

public:
    User(const string& uname, const string& pwd);
    string getUsername() const;
    string getPassword() const;
};

#endif
```

```
#ifndef AUTH_H
#define AUTH_H

#include "User.h"

class Auth {
public:
    bool login(const User& user, const string& inputPassword);
};

#endif
```

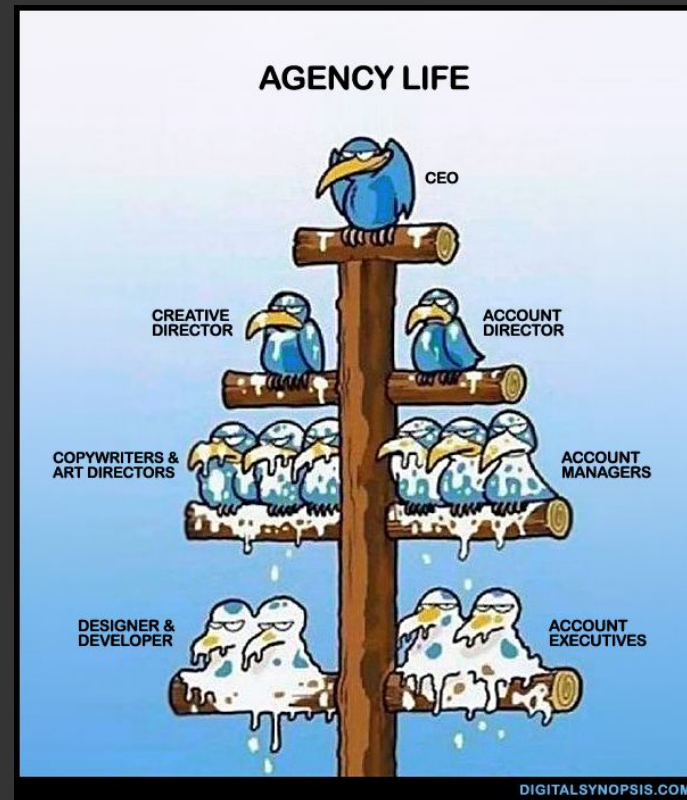
```
int main() {
    User user("alice", "1234");
    Auth auth;

    if (auth.login(user, "1234"))
        std::cout << "Login successful!" << std::endl;
    else
        std::cout << "Login failed!" << std::endl;

    return 0;
}
```

Hierarchy

- **What it is:** Organizing classes in a parent-child relationship using inheritance.
- **Purpose:** Promotes reuse and logical structure of related classes.
- **Example:** Animal → Mammal → Dog.



Hierarchy

```
#include <iostream>
using namespace std;

class Animal {
public:
    void eat() {
        cout << "Animal eats food." << endl;
    }

    void sleep() {
        cout << "Animal sleeps." << endl;
    }
};
```

```
class Mammal : public Animal {
public:
    void breathe() {
        cout << "Mammal breathes air." << endl;
    }
};
```

```
class Dog : public Mammal {
public:
    void bark() {
        cout << "Dog barks." << endl;
    }
};
```

SOLID Reminder

Letter	Principle	Focus
S	Single Responsibility Principle	Class should do one thing
O	Open/Closed Principle	Open for extension, closed for modification
L	Liskov Substitution Principle	Subtypes must be substitutable
I	Interface Segregation Principle	Many small interfaces > one big one
D	Dependency Inversion Principle	Depend on abstractions, not concrete classes

SOLID to OOP

S – Single Responsibility Principle	Encapsulation, Modularity	Encapsulates behavior and keeps modules focused on one reason to change.
O – Open/Closed Principle	Abstraction, Modularity	Promotes designing via abstract interfaces that allow extension without modifying code.
L – Liskov Substitution Principle	Hierarchy (Inheritance)	Ensures that subclasses maintain the behavior contract of their base class.
I – Interface Segregation Principle	Abstraction	Promotes fine-grained interfaces so clients aren't forced to depend on what they don't use.
D – Dependency Inversion Principle	Abstraction, Modularity	High-level modules depend on abstractions, not concretions; promotes decoupling of modules.

SOLID to OOP

OOP Principle	Helps Fulfill These SOLID Principles
Abstraction	SRP, OCP, ISP, DIP
Encapsulation	SRP
Modularity	SRP, OCP, DIP
Hierarchy	LSP

**ME WHEN THE TEACHER
USES ME AS A BAD EXAMPLE**



imgflip.com

memegenerator.com

Example 1

```
class WeatherService {  
public:  
    std::string fetchWeatherJson(const std::string& url) {  
        // Leaking low-level HTTP to the caller  
        CURL* curl = curl_easy_init();  
        // ... full HTTP handling here ...  
        return rawJson;  
    }  
};
```

Example 2

```
class Invoice {  
public:  
    std::vector<LineItem> items;  
    double total;  
  
    void recalculateTotal() {  
        total = 0;  
        for (auto& item : items)  
            total += item.price * item.quantity;  
    }  
};
```

Example 3

```
class App {  
public:  
    void renderUI();  
    void handleUserInput();  
    void queryDatabase();  
    void sendAnalytics();  
    void saveConfig();  
};
```

Example 4

```
#include <iostream>
#include <string>

class Logger {
public:
    void log(const std::string& message) {
        std::cout << "LOG: " << message << std::endl;
    }
};
```

```
#include "Logger.h"

class Button : public Logger {
public:
    void click() {
        log("Button clicked");
        // Other click logic
    }
};
```

```
#include "Button.h"

int main() {
    Button b;
    b.click();
    return 0;
}
```


Example 5

```
class Rectangle {
protected:
    int width;
    int height;

public:
    void setWidth(int w) {
        width = w;
    }

    void setHeight(int h) {
        height = h;
    }

    int getWidth() const {
        return width;
    }

    int getHeight() const {
        return height;
    }

    int getArea() const {
        return width * height;
    }
};
```

Example 5

```
#include "Rectangle.h"

class Square : public Rectangle {
public:
    void setWidth(int w) {
        width = w;
        height = w;
    }

    void setHeight(int h) {
        width = h;
        height = h;
    }
};
```

Example 5

```
#include <iostream>
#include "Square.h"

int main() {
    Rectangle* shape = new Square();
    shape->setWidth(5);
    shape->setHeight(10);

    std::cout << "Expected area: 50" << std::endl;
    std::cout << "Actual area: " << shape->getArea() << std::endl;

    delete shape;
    return 0;
}
```

Example 6

```
#include <iostream>
#include <vector>
#include <fstream>
#include <string>

class Application : public std::vector<std::string> {
public:
    double userBalance;
    std::string configPath;
    std::string dbConnectionString;

    void process() {
        for (auto& name : *this) {
            std::cout << "Processing user: " << name << std::endl;
            userBalance += 100;
            std::ofstream file(configPath);
            file << "Updated config for " << name << std::endl;
            file.close();
        }
    }

    void reset() {
        clear();
        userBalance = 0;
        std::remove(configPath.c_str());
    }
};
```

Example 7

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>

class OrderProcessor {
public:
    std::string customerName;
    std::vector<std::string> items;
    double totalAmount;
    bool isPaid;

    void loadOrderFromFile(const std::string& filename) {
        std::ifstream file(filename);
        std::getline(file, customerName);
        std::string item;
        while (std::getline(file, item)) {
            items.push_back(item);
        }
        file.close();
    }

    void calculateTotal() {
        totalAmount = 0;
        for (auto& item : items) {
            totalAmount += 9.99; // flat price per item
        }
    }
};
```

```
void pay() {
    isPaid = true;
    std::cout << "Payment completed for: " << customerName << std::endl;
}

void generateReport() {
    std::cout << "Order for " << customerName << ":\n";
    for (auto& item : items)
        std::cout << "- " << item << std::endl;
    std::cout << "Total: $" << totalAmount << " | Paid: " << (isPaid ? "Yes" : "No")
};
```

The background of the entire image is a dark, stormy sky with vibrant red lightning bolts and glowing clouds. In the top right corner, there is a small, faint white icon of a hand pointing downwards.

N

DESIGN PATTERNS

COMING SOON TO THE
WHITEBOARD NEAR YOU!

Оценить это изображение

